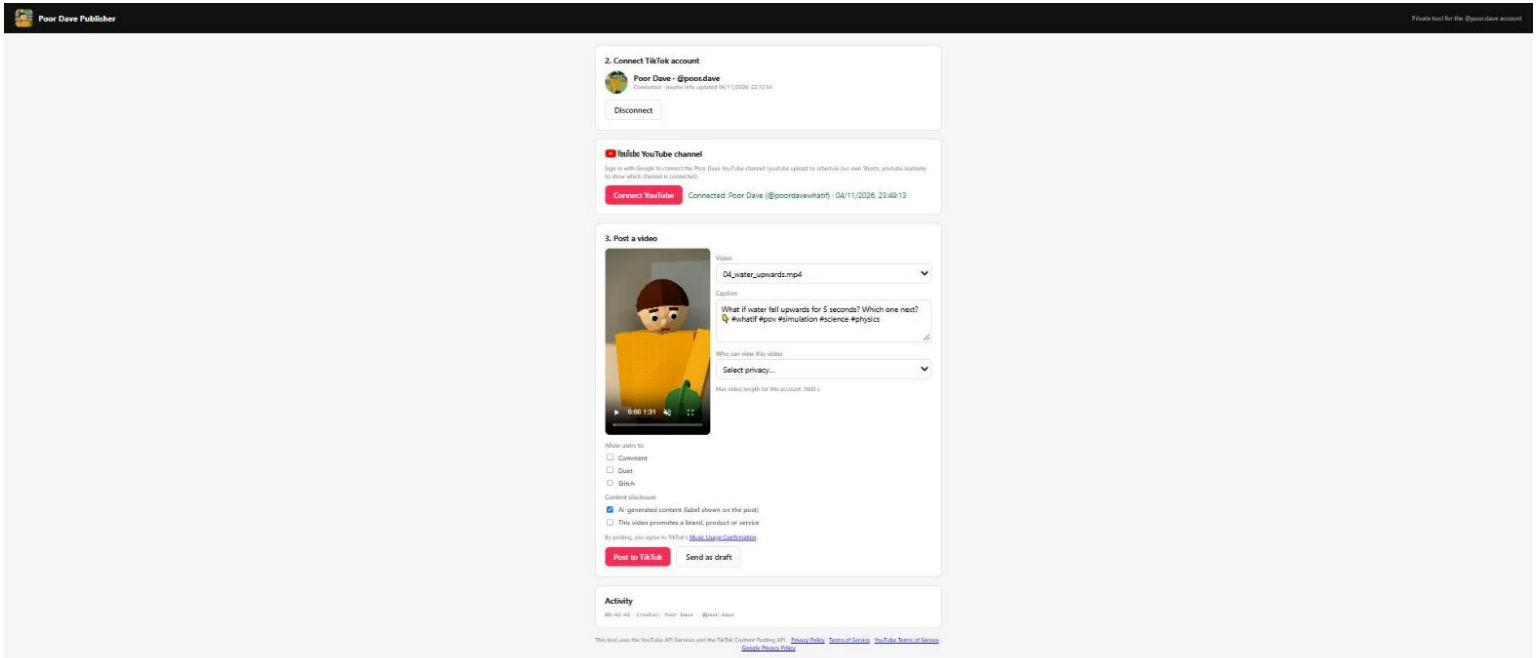
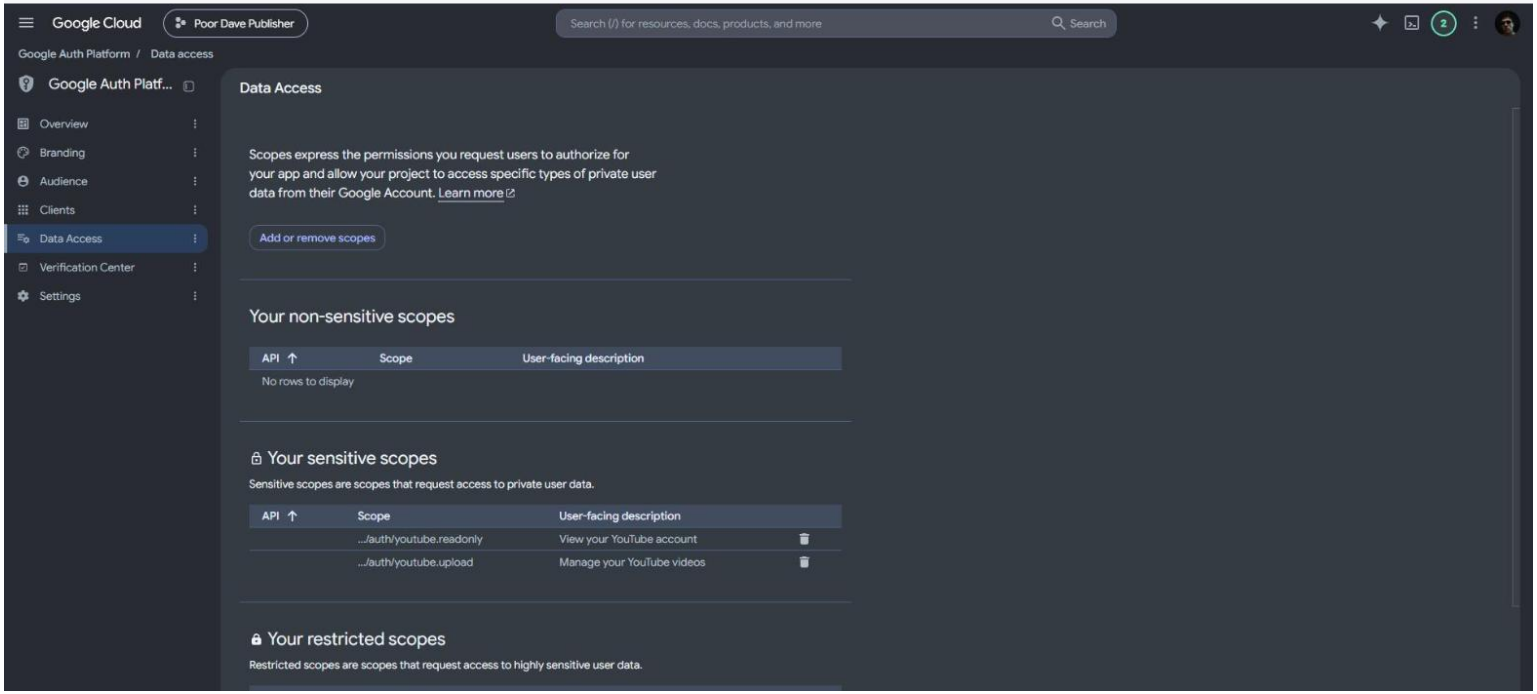
Poor Dave Publisher - OAuth scopes, upload interface and upload code (project poor-dave-publisher)





Upload flow (scripts/publish.py, run by GitHub Actions; full source: github.com/Vitrella/poor-dave-media)

1. The owner clicks "Connect YouTube" on publisher.html, signs in with Google and grants youtube.upload + youtube.readonly (OAuth consent).
2. The authorization code is exchanged server-side (GitHub Actions); tokens are stored AES-256 encrypted. channels.list confirms the channel (Poor Dave, @poordavewhatif).
3. For each scheduled video in queue.json, videos.insert (resumable upload) sends our own MP4 as private with publishAt, selfDeclaredMadeForKids=false, containsSyntheticMedia=true.
4. Revocation: myaccount.google.com/permissions or security.google.com/settings/security/permissions; tokens are then deleted.

```
def youtube_profile(tokens):
    """Save the connected channel (no secrets) for the publisher page."""
    st, j, _ = http('GET', 'https://www.googleapis.com/youtube/v3/channels?part=snippet&mine=true',
                    headers={'Authorization': 'Bearer ' + google_access(tokens)})
    items = j.get('items') or []
    if not items:
        raise RuntimeError(f'YouTube channels: {j}')
    sn = items[0]['snippet']
    save(os.path.join(STATUS, 'youtube_account.json'), {
        'channel_id': items[0]['id'], 'title': sn.get('title'), 'handle': sn.get('customUrl'),
        'refresh_token': bool(tokens.get('youtube', {}).get('refresh_token')), 'updated': now().isoformat()})


def youtube_upload(tokens, item, video):
    meta = {'snippet': {'title': item['title'], 'description': item['caption'], 'categoryId': '28',
                        'defaultLanguage': 'en', 'defaultAudioLanguage': 'en'},
            'status': {'privacyStatus': 'private', 'publishAt': item['youtube_at'],
                       'selfDeclaredMadeForKids': False, 'containsSyntheticMedia': True}}
    auth = {'Authorization': 'Bearer ' + google_access(tokens)}
    st, j, h = http('POST', 'https://www.googleapis.com/upload/youtube/v3/videos?uploadType=resumable&part=snippet,status',
                    raw=json.dumps(meta).encode(), headers={**auth, 'Content-Type': 'application/json; charset=UTF-8',
                                                            'X-Upload-Content-Type': 'video/mp4',
                                                            'X-Upload-Content-Length': str(len(video))})
    loc = h.get('Location') or h.get('location')
    if not loc:
        raise RuntimeError(f'YouTube init HTTP {st}: {j}')
    st, j, _ = http('PUT', loc, raw=video, headers={**auth, 'Content-Type': 'video/mp4'})
    if 'id' not in j:
        raise RuntimeError(f'YouTube upload HTTP {st}: {j}')
    return {'video_id': j['id'], 'url': 'https://youtube.com/shorts/' + j['id']}


# ---------------- main ----------------
def run():
```